Iterator

Iterator · Round-robin assign jobs to workers · Build with Waqar

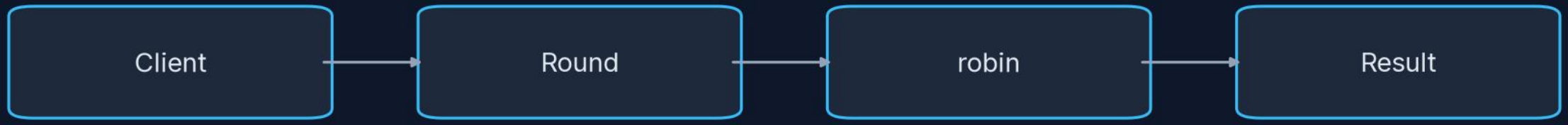
Today's Hook

- Problem: Round-robin assign jobs to workers
- Pattern: Iterator — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Round-robin assign jobs to workers
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Iterator: Round-robin assign jobs to workers

Real-World Scenario

- Running example: Round-robin assign jobs to workers
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.iterator.problem;

import java.util.*;

public final class NaiveAssign {
  private int i; private final List<String> w;
  public NaiveAssign(List<String> w){ this.w=w; }
  public String next(){ return w.get(i++ % w.size()); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.iterator.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

final class RoundRobin implements Iterable<String> {
  private final List<String> workers; private int i;
  RoundRobin(List<String> workers){ this.workers=List.copyOf(workers); }
  public Iterator<String> iterator(){
    return new Iterator<>(){
      public boolean hasNext(){ return !workers.isEmpty(); }
      public String next(){ if(workers.isEmpty()) throw new NoSuchElementException();
        String w=workers.get(i % workers.size()); i++; return w; }
    };
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.iterator.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Iterator: round-robin workers ===");
    System.out.println("--- naive ---");
var n=new NaiveAssign(List.of("w1","w2")); System.out.println(n.next()+" "+n.next()+" "+n.next());
System.out.println("--- iterator ---");
var it=new RoundRobin(List.of("w1","w2")).iterator();
System.out.println(it.next()+" "+it.next()+" "+it.next());
  }
}
```

Walkthrough Output

```
=== Iterator: round-robin workers ===
--- naive ---
w1 w2 w1
--- iterator ---
w1 w2 w1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Iterator.

Related Patterns

- Contrast Iterator with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Iterator → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.